

Documentación del lenguaje MiniLex

1. Introducción

MiniLex es un analizador léxico gráfico desarrollado con Electron y FLEX. Reconoce los componentes de un lenguaje sencillo creado para realizar pruebas y presenta los tokens encontrados en el código de entrada.

2. Descripción del lenguaje MiniLex

MiniLex es un lenguaje de prueba diseñado para verificar el funcionamiento del analizador léxico. Sus patrones se definen en `lexer/lexer.l`. El análisis identifica palabras, números y símbolos; no comprueba la sintaxis ni ejecuta el código. Las mayúsculas y minúsculas se distinguen.

3. Elementos reconocidos

Elemento	Regla y ejemplos
Palabras reservadas	<code>int</code> , <code>float</code> , <code>string</code> , <code>bool</code> , <code>if</code> , <code>else</code> , <code>while</code> , <code>return</code> y <code>print</code> .
Identificadores	Patrón: <code>[a-zA-Z][a-zA-Z0-9_]*</code> . Empiezan con letra de A a Z, de a a z o guion bajo; después admiten también dígitos. Ejemplos: <code>edad</code> , <code>_nota</code> , <code>dato2</code> .
Números enteros	Patrón: <code>[0-9]+</code> . Uno o más dígitos. Ejemplos: <code>20</code> , <code>70</code> .
Números decimales	Patrón: <code>[0-9]+\.[0-9]+</code> . Dígitos a ambos lados del punto. Ejemplo: <code>85.5</code> . El signo menos se reconoce por separado.
Cadenas	Texto entre comillas dobles, incluso vacío, sin salto de línea. Se admite una barra inversa seguida de un carácter distinto de salto de línea. Ejemplos: <code>"Erick"</code> , <code>""</code> , <code>"Hola\""</code> . El lexer reconoce el texto sin interpretar los escapes.
Booleanos	<code>true</code> y <code>false</code> se reconocen como <code>BOOLEANO</code> .
Operadores aritméticos	<code>+</code> (SUMA), <code>-</code> (RESTA), <code>*</code> (MULTIPLICACION), <code>/</code> (DIVISION) y <code>%</code> (MODULO).
Operadores relacionales	<code>>=</code> (MAYOR_IGUAL), <code><=</code> (MENOR_IGUAL), <code>==</code> (IGUALDAD), <code>!=</code> (DIFERENTE), <code>></code> (MAYOR_QUE) y <code><</code> (MENOR_QUE).
Operadores lógicos	<code>&&</code> (AND), <code> </code> (OR) y <code>!</code> (NOT).
Asignación	<code>=</code> (ASIGNACION). Ejemplo: <code>edad = 20;</code> .
Paréntesis	<code>(</code> y <code>)</code> : <code>PARENTESIS_IZQUIERDO</code> y <code>PARENTESIS_DERECHO</code> .
Llaves	<code>{</code> y <code>}</code> : <code>LLAVE_IZQUIERDA</code> y <code>LLAVE_DERECHA</code> .
Corchetes	<code>[</code> y <code>]</code> : <code>CORCHETE_IZQUIERDO</code> y <code>CORCHETE_DERECHO</code> .
Punto y coma	<code>;</code> (<code>PUNTO_COMA</code>).
Coma	<code>,</code> (<code>COMA</code>).
Comentarios	Comienzan con <code>//</code> y terminan al finalizar la línea. Se ignoran. Ejemplo: <code>// comentario</code> . No existe una regla para comentarios de bloque.
Errores léxicos	Cada carácter que no coincide con las reglas anteriores genera <code>ERROR</code> . Ejemplos: <code>@</code> y <code>#</code> fuera de cadenas y comentarios.

Los espacios, tabulaciones y retornos de carro se ignoran. Los saltos de línea actualizan el número de línea y no generan tokens.

4. Código utilizado para las pruebas

Se utilizó el siguiente código para comprobar el reconocimiento de palabras reservadas, identificadores, valores y símbolos:

```
int edad = 20;
float nota = 85.5;
string nombre = "Erick";

if (nota >= 70) {
    print("Aprobado");
}
```

5. Resultado del análisis

El programa muestra Token, Lexema y Línea. Token indica la categoría reconocida; Lexema es el texto encontrado; Línea señala su ubicación en el código. Al ejecutar esta prueba con lexer.exe se reconocieron 28 tokens sin errores léxicos.

Algunos ejemplos del resultado son:

```
PALABRA_RESERVADA → int
IDENTIFICADOR → edad
ASIGNACION → =
ENTERO → 20
DECIMAL → 85.5
CADENA → "Erick"
MAYOR_IGUAL → >=
```

6. Prueba de errores léxicos

Los caracteres no definidos por el lenguaje, como @ y #, se reconocen como ERROR cuando aparecen fuera de una cadena o de un comentario. Por ejemplo, la entrada siguiente produce dos errores léxicos en la línea 1:

```
@ #
```

```
ERROR → @ → línea 1
ERROR → # → línea 1
```

7. Conclusión

Las pruebas permiten comprobar que FLEX reconoce los patrones establecidos en lexer.l, clasifica los componentes del código y detecta los caracteres no admitidos. El ejemplo válido y la prueba de errores verifican ambos comportamientos.